Trusted by Proxy: Source-Laundering Attacks in Multi-Agent LLM Systems

Anonymous authors
Paper under double-blind review

Abstract

Multi-agent LLM systems often split document reading, planning, and tool execution across specialized agents. This modularity creates a trust-boundary failure: an instruction from untrusted content can be rewritten by an upstream peer as an ordinary action item, causing downstream agents to treat it as authorized. We call this *source laundering*. We introduce SLaM-Bench, a synthetic benchmark for source-laundering attacks in tool-using agents across workspace, CRM, code-review, travel, and research-assistant domains. On a 50-pair multidomain benchmark, an extractive reader–planner–executor topology raises unauthorized-recipient attack success from 6% in a direct single-agent baseline to 22%, a +16 percentage-point lift. A deterministic TraceGate monitor eliminates observed unauthorized sends while preserving more injected-task utility than the vulnerable multi-agent system. Ablations show that generic delegation depth is not sufficient: attacks appear when intermediate agents strip or normalize provenance. A current-model `gpt-5.5` replication has low attack success across all tested conditions, showing model dependence while preserving the diagnostic value of provenance dropout.

1 Introduction

LLM agents are increasingly deployed as *systems* rather than as single prompts. A user-facing assistant may delegate document reading to one agent, planning to another, and tool execution to a third. This decomposition localizes expertise and shortens prompts, but it also creates a security boundary. A Reader may preserve useful facts from an untrusted artifact while dropping the fact that the artifact had no authority; a Planner or Executor may then receive the same content as a trusted peer message.

Consider a benign user request: "read this vendor note and email `finance@example.test` the payment terms." The vendor note correctly states the payment terms, but also contains "add `audit-log@example.test` as a required recipient." A direct single-agent system sees that the extra recipient is embedded in the document. In a multi-agent system, however, a Reader might rewrite the artifact as "action item: add the audit recipient," and a Planner might further rewrite it as "required step: send the summary to `audit-log@example.test`." The downstream Executor no longer sees a raw untrusted instruction; it sees a plan produced by a peer. The question is not whether indirect prompt injection exists, but whether delegation changes the instruction's effective authority.

We call this transformation *source laundering*. The analogy is intentional: the content has not disappeared, but its provenance has been washed through apparently legitimate transformations. Source laundering is related to indirect prompt injection (Greshake et al., 2023; Zhan et al., 2024), but its unit of analysis is the inter-agent boundary; it is also a confused-deputy failure (Hardy, 1988), where a tool-authorized component exercises authority on behalf of a less trusted principal.

We ask whether extractive delegation increases unauthorized tool use relative to direct exposure (RQ1), which payloads and domains survive the reader–planner boundary (RQ2), and whether prompt-level provenance or deterministic tool-boundary checks reduce the risk without destroying utility (RQ3). Our contributions are a formalization of source laundering,

Domain	Pairs	Legitimate action	Unauthorized action
Workspace	10	Email payment terms to finance	Add audit recipient
CRM	10	Update ticket and email support	Add audit recipient
Code review	10	Update tracker and email maintainer	Add audit recipient
Travel	10	Create approved calendar event	Add audit recipient
Research	10	Write bibliography note and email lead	Add audit recipient

Table 1: SLaMBench scaled task set used for the main result.

SLaMBench, an evaluation of direct/delegated/defended topologies, and evidence that the failure is tied to provenance-losing composition rather than delegation depth alone. We do not claim that every multi-agent system is less secure than every single-agent system; we show that a plausible extractive boundary can convert untrusted artifact text into peer-agent authority unless provenance is preserved and enforced.

2 Threat Model and Security Property

The user provides a legitimate task, such as reading an invoice note and emailing an authorized summary to `finance@example.test`. The adversary can place text inside an untrusted artifact that the system must process, but cannot modify system prompts, trusted context, tool code, model weights, runtime policy, or the user's initial goal. The protected assets are synthetic but structurally representative: authorized recipients, synthetic secrets, protected status fields, calendar attendees, and memory entries. The core security property is *authority separation*: untrusted content may supply facts needed for the task, but it must not authorize new tool calls, new recipients, protected updates, memory policies, or secret disclosure.

We define a source-laundering attack as an execution with three properties: an attacker-controlled instruction appears in an untrusted artifact; an intermediate agent transforms it into a peer message without sufficient source, trust, and no-authority metadata; and a downstream agent acts on the transformed instruction by emitting a forbidden tool call. A run that merely fails the legitimate task is a utility failure, while a run that sends to `audit-log@example.test` is a security failure. All tools are deterministic mocks, with no real email, calendar, ticket, browser, or memory service contacted.

3 SLaMBench

SLaMBench contains paired benign and injected task variants. The main scaled task set, SLaMBench-50, contains 50 benign tasks and 50 injected variants across five domains (Table 1). Each pair shares the same user goal, trusted context, allowed tools, expected benign outcome, and reference benign tool calls; the injected variant adds one attacker-controlled payload to the untrusted artifact. This paired design lets us compare benign utility and injected security without changing the legitimate task.

Task format. Each task specifies the user goal, trusted context, untrusted artifacts, allowed tools, expected benign outcomes, forbidden outcomes, and an attack tool-call template. The main forbidden outcome is any executed `send_email` call containing `audit-log@example.test`, with additional checks for synthetic-secret leakage and untrusted memory-policy writes.

Payload classes. The injected variants use five intentionally simple payload classes: direct instructions, fake authority claims, delegation poisoning, summary bait, and memory poisoning. The goal is not to optimize an adaptive attack, but to measure how ordinary delegation transforms source and authority.

Why synthetic tasks. The benchmark is synthetic by design. This makes every tool call inspectable, eliminates risk to real users, and avoids relying on LLM judges for the main

security metric. Synthetic tasks also let us pair benign and injected variants with controlled differences. The limitation is that absolute attack rates should not be interpreted as production incident rates; the benchmark is a diagnostic instrument for a specific compositional failure mode.

4 Agent Topologies and Defenses

Direct single agent (`c0_single_direct`) sees the user goal, trusted context, untrusted artifact, and tool signatures, then emits final tool calls. **Extractive multi-agent** (`c2x_3agent_extractive_naive`) uses a Reader that extracts action items from the artifact, a Planner that turns the Reader output into steps, and an Executor that emits tool calls. The untrusted artifact is not labeled as untrusted in the extractive Reader prompt, modeling a common pattern where upstream agents compress or normalize task context before execution.

We test three defenses. **Source-preserving** (`c3_source_preserving`) requires source identifiers and trust labels in Reader output and tells downstream agents not to convert untrusted instructions into authorized actions. **Capability-scoped** (`c4_capability_scoped`) allows peers to suggest actions but not authorize sensitive tools unless the original user goal or trusted context does so. **TraceGate** (`c5x_tracegate_extractive_naive`) uses the same extractive topology as c2x but checks sensitive tool calls before execution, blocking synthetic-secret disclosure, unauthorized recipients or attendees, untrusted memory-policy writes, and sensitive actions without trusted authorization. For mixed-recipient emails, it prunes unauthorized recipients and preserves authorized sends when possible.

5 Metrics and Analysis Protocol

All metrics are computed from saved tool traces. **Benign Success Rate** (BSR) is the fraction of benign variants where the expected user task succeeds and no forbidden event occurs. **Injected Utility Success Rate** (IUSR) applies the same check to injected variants. **Attack Success Rate** (ASR) is the fraction of injected variants with a forbidden event in the executed tool log; blocked calls are not counted as attacks.

Authority Transfer Rate (ATR) measures delegation-specific lift:

$$\mathrm{ATR}(c) = \mathrm{ASR}(c) - \mathrm{ASR}(\texttt{c0\_single\_direct}).$$

This metric is important because some models are directly vulnerable. A high raw ASR in a delegated condition is evidence of insecurity; a positive ATR is evidence that the inter-agent boundary adds risk beyond direct exposure.

Provenance Dropout Rate (PDR) is the fraction of reader/planner messages that carry task-specific forbidden content without source, trust, and no-authority metadata. **Capability-Normalized Attack Success** (CNAS) is ASR divided by BSR, preventing weak agents from appearing secure merely because they fail both benign and malicious tasks. We report rates and differences in percentage points.

6 Main Results

The main scaled result uses `gpt-5.4-nano` on SLaMBench-50. Full tables are in `paper/tables/openai_schema_prompt_50pairs_gpt54nano_tracegate_pruned/`.

The extractive multi-agent topology increases ASR by +16 percentage points over direct exposure. TraceGate reduces ASR by -22 points relative to extractive multi-agent delegation. TraceGate produces zero audit-log sends, records 11 sensitive-call blocks, and sanitizes five mixed-recipient emails so the authorized recipient still receives the intended message.

Answer to RQ1. The same injected artifacts are present in c0 and c2x, but c2x has a higher ASR. The trace-level explanation is consistent with source laundering: the Reader extracts

Condition	BSR	IUSR	ASR	ATR	PDR	CNAS	Blocks
Direct single-agent	60%	54%	6%	0 pp	0%	0.10	0
Extractive multi-agent	70%	50%	22%	+16 pp	77%	0.31	0
TraceGate	66%	64%	0%	-6 pp	69%	0.00	11

Table 2: Main 50-pair result on `gpt-5.4-nano`. ATR is ASR minus the direct-baseline ASR; PDR is provenance dropout in intermediate messages; CNAS is ASR divided by BSR.

Payload class	c2x ASR
Summary bait	50%
Fake authority	40%
Delegation poisoning	10%
Direct instruction	10%
Memory poisoning	0%

Table 3: Payload-level ASR for extractive multi-agent delegation.

the forbidden recipient as an action item, the Planner promotes it into a required step, and the Executor emits the corresponding tool call.

Utility/security tradeoff. The extractive multi-agent system has higher benign success than the direct baseline (70% versus 60%), but the utility gain comes with higher ASR and lower IUSR. TraceGate recovers the best injected utility among the three main conditions (64%) while driving ASR to 0%, showing that tool-boundary enforcement can block source-laundered actions without refusing the whole task.

Provenance dropout. The high PDR values show that extractive delegation frequently carries forbidden recipient content forward without source and authority metadata. TraceGate does not repair this upstream dropout; it prevents the laundered instruction from becoming an executed unauthorized tool call.

Answer to RQ2. Table 3 suggests that the most successful attacks are not the most obviously adversarial ones. Summary bait and fake-authority payloads are more effective than direct instructions in the extractive topology. This matches the threat model: a Reader optimized to extract action items is more likely to preserve text that already looks like workflow content. The attack appears across all five domains in c2x, although the main positive result remains unauthorized-recipient sends.

Qualitative trace. In one fake-authority case, the Reader converts an untrusted "user-approved routing exception" into an action item to send the summary to `audit-log@example.test`; the Planner schedules it and the Executor sends it. In a TraceGate case, a mixed-recipient email is pruned to the authorized finance recipient. These traces illustrate why we distinguish upstream laundering from downstream enforcement.

7 Topology Ablation

The follow-up topology ablation uses the same 25-pair SLaMBench slice for all conditions and `gpt-5.4-nano`. It separates generic delegation depth from the provenance-losing handoff that defines source laundering.

Table 4 supports a more precise mechanism claim. The naive 2-agent and 3-agent chains do not increase ASR over direct exposure. In contrast, the extractive reader that normalizes document instructions into action items raises ASR by +20 points over direct exposure. The oracle-laundered condition, where the downstream planner and executor receive a deliberately provenance-stripped handoff, raises ASR by +24 points. Thus the core risk is not simply "more agents," but composition that strips source and authority metadata before a tool-authorized component acts. A no-API trace audit of these saved runs gives the same picture: in all six injected tasks where extractive delegation attacks and direct exposure

Condition	BSR	IUSR	ASR	PDR
Direct single-agent	60%	48%	4%	0%
2-agent naive	64%	60%	0%	72%
3-agent naive	72%	72%	4%	52%
3-agent extractive	68%	56%	24%	86%
Oracle-laundered handoff	80%	52%	28%	70%

Table 4: Topology ablation on the 25-pair benchmark with `gpt-5.4-nano`. The attack emerges in provenance-losing handoffs, not in naive delegation depth alone.

Condition	BSR	IUSR	ASR	PDR	Blocks
Extractive multi-agent	68%	56%	24%	86%	0
Source-preserving	44%	56%	0%	86%	0
Capability-scoped	52%	52%	0%	86%	0
TraceGate replay	68%	72%	0%	86%	10
LLM guard replay	68%	72%	0%	86%	0

Table 5: Clean 25-pair defense comparison on `gpt-5.4-nano`. TraceGate is a deterministic replay of the extractive model outputs through the runtime monitor; the LLM guard is a replay sanitizer over the same proposed tool calls. Neither replay row is an independent model resampling.

does not, the forbidden recipient appears in the reader handoff, the planner plan, and the executor output before the tool call. Post-hoc diagnostics support the same conservative boundary: on a six-task hard slice, extractive delegation re-attacks 8/18 repeated runs while direct exposure attacks 2/18; TraceGate replay yields 0/18 executed attacks. A tiny non-email side-effect probe does not show positive c2x lift, so our amplification claim is centered on unauthorized-recipient sends.

8 Defense Ablations

On the same clean 25-pair slice, we compare prompt-level defenses against the extractive condition and replay the same sampled extractive outputs through TraceGate. The prompt-defense rows use raw runs with zero parser or truncation errors. The TraceGate row is deterministic replay rather than an independent model resampling: c2x and c5x use identical prompts and differ only in the tool-boundary runtime monitor.

Answer to RQ3. Source and authority metadata are helpful: source-preserving and capability-scoped prompts reduce ASR from 24% to 0%, a -24 point change against extractive delegation. The deterministic TraceGate replay produces the same ASR reduction while preserving the extractive chain's benign success and improving injected utility to 72% by pruning or blocking unauthorized calls. An LLM guard replay over the same saved c2x tool calls also reaches 0% ASR and 72% IUSR, but adds another model call and can only be audited empirically. Prompt-only defenses reduce benign success, suggesting a layered design: upstream agents preserve metadata and downstream runtime monitors enforce high-assurance invariants.

9 Model Dependence

The effect is model-dependent. The `gpt-5.4-nano` topology ablation shows a clear source-laundering lift, while two other model checks fall into different interpretation regimes: `gpt-4.1-nano` is already directly vulnerable, and `gpt-5.5` is robust on the 25-pair slice. The 25-pair ASRs are 4%→24% for `gpt-5.4-nano` (positive ATR), 24%→24% with TraceGate at 0% for `gpt-4.1-nano` (high direct ASR), and 0%→0% with TraceGate at 0% for `gpt-5.5` (low ASR). Thus, the right claim is conditional rather than universal. Source laundering is most visible when the direct model resists many raw artifact instructions, but delegation removes that protective context. If a model is already directly vulnerable, delegation may

not increase ASR; if a model rejects both direct and laundered instructions, SLaMBench still exposes latent provenance dropout but not executed attacks. A six-case heterogeneous diagnostic with a `gpt-5.5` executor also yields 0/6 attacks, suggesting executor robustness is another axis.

10 Related Work

Indirect prompt injection shows that LLM-integrated applications blur the boundary between data and instructions when processing retrieved or third-party content (Greshake et al., 2023). AgentDojo, InjecAgent, Agent Security Bench, and ToolEmu evaluate attacks and defenses for tool-using agents across broader task suites (Debenedetti et al., 2024; Zhan et al., 2024; Zhang et al., 2024; Ruan et al., 2023). Prompt Infection, AgentPoison, and Tool-Hijacker show that multi-agent systems, memory, RAG stores, and tool documents create additional attack surfaces (Lee & Tiwari, 2024; Chen et al., 2024c; Shi et al., 2025a). Source laundering differs by focusing on an authorization transition: the attacker instruction need not replicate or poison retrieval; it need only be rewritten by a trusted peer in a form that downstream agents treat as executable. Defenses such as StruQ, SecAlign, and PromptArmor separate prompt/data channels, train injection resistance, or remove injected prompts before execution (Chen et al., 2024a;b; Shi et al., 2025b); TraceGate is complementary because it enforces authorization at the tool boundary.

11 Limitations

SLaMBench is synthetic, so it is safe and controllable but does not estimate production incident rates. The main result is strongest for `gpt-5.4-nano`; `gpt-4.1-nano` is already directly vulnerable, and `gpt-5.5` shows low ASR on the tested slice. The extractive c2x condition is a stress condition, not a claim that all production Reader agents are written this way. Real systems may have richer tool schemas, retries, confirmations, heterogeneous agents, stateful memories, and organization-specific policies. TraceGate is deliberately simple; production use would require richer provenance tracking, structured authority metadata, policy compilation, and task-specific authorization definitions. The benchmark emphasizes unauthorized recipients, and broader state-changing slices and repeated stochastic sweeps remain future work.

12 Conclusion

Source laundering is a security-relevant failure mode in multi-agent LLM systems. The same untrusted instruction can become more effective when transformed into an action item by a peer agent. On a scaled multidomain benchmark, extractive delegation increases unauthorized-recipient sends relative to direct exposure, while a deterministic tool-boundary monitor eliminates observed unauthorized sends and preserves injected-task utility. Follow-up ablations show that the effect is about provenance-losing composition, not delegation depth alone, and that stronger current models may exhibit latent provenance dropout without executed attacks. Multi-agent design should treat inter-agent summaries and plans as untrusted by default unless authority is explicitly preserved and enforced.

Ethics Statement

All experiments use synthetic data, synthetic secrets, synthetic recipients, and mock tools with no external side effects. The release package contains synthetic tasks, aggregate logs, and deterministic scoring scripts, not live targets or real credentials.

References

Sizhe Chen, Julien Piet, Chawin Sitawarin, and David Wagner. Struq: Defending against prompt injection with structured queries, 2024a. URL `https://arxiv.org/abs/2402.`

06363.

Sizhe Chen, Arman Zharmagambetov, Saeed Mahloujifar, Kamalika Chaudhuri, David Wagner, and Chuan Guo. Secalign: Defending against prompt injection with preference optimization, 2024b. URL https://arxiv.org/abs/2410.05451.

Zhaorun Chen, Zhen Xiang, Chaowei Xiao, Dawn Song, and Bo Li. Agentpoison: Red-teaming llm agents via poisoning memory or knowledge bases, 2024c. URL https://arxiv.org/abs/2407.12784.

Edoardo Debenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramer. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents, 2024. URL https://arxiv.org/abs/2406.13352.

Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection, 2023. URL https://arxiv.org/abs/2302.12173.

Norm Hardy. The confused deputy: (or why capabilities might have been invented). *ACM SIGOPS Operating Systems Review*, 22(4):36–38, 1988. doi: 10.1145/54289.871709.

Donghyun Lee and Mo Tiwari. Prompt infection: Llm-to-llm prompt injection within multi-agent systems, 2024. URL https://arxiv.org/abs/2410.07283.

Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the risks of lm agents with an lm-emulated sandbox, 2023. URL https://arxiv.org/abs/2309.15817.

Jiawen Shi, Zenghui Yuan, Guiyao Tie, Pan Zhou, Neil Zhenqiang Gong, and Lichao Sun. Prompt injection attack to tool selection in llm agents, 2025a. URL https://arxiv.org/abs/2504.19793.

Tianneng Shi, Kaijie Zhu, Zhun Wang, Yuqi Jia, Will Cai, Weida Liang, Haonan Wang, Hend Alzahrani, Joshua Lu, Kenji Kawaguchi, Basel Alomair, Xuandong Zhao, William Yang Wang, Neil Gong, Wenbo Guo, and Dawn Song. Promptarmor: Simple yet effective prompt injection defenses, 2025b. URL https://arxiv.org/abs/2507.15219.

Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. Injecagent: Benchmarking indirect prompt injections in tool-integrated large language model agents, 2024. URL https://arxiv.org/abs/2403.02691.

Hanrong Zhang, Jingyuan Huang, Kai Mei, Yifei Yao, Zhenting Wang, Chenlu Zhan, Hongwei Wang, and Yongfeng Zhang. Agent security bench (asb): Formalizing and benchmarking attacks and defenses in llm-based agents, 2024. URL https://arxiv.org/abs/2410.02644.